

Comparative Analysis of Machine Learning Models for Plant Species Identification

Marley Williams
School of Engineering
University of the West of England (UWE)
Bristol, UK
marley2.williams@live.uwe.ac.uk
Student ID: 23031960

Abstract - This project presents a comparative analysis of five machine learning model architectures deployed on a Raspberry Pi-powered smart glasses platform for real-time plant species identification. Machine vision and edge computing are increasingly applied within agricultural and environmental monitoring contexts, with wearable devices offering promising potential for accessible, on-device plant identification in the field. However, real-time machine vision tends to require large processing power to execute accurately, whereas wearable edge hardware is constrained by limited battery and memory capacity, and understanding how to bridge this gap is essential to future developments within this field. A quantitative methodology was employed, training and evaluating five architectures, a custom convolutional neural network, MobileNetV2, EfficientNet, ResNet50, and InceptionV3, on a publicly available dataset of seven plant species comprising 816 augmented leaf images, with each model assessed across accuracy, precision, recall, inference speed, and model size before being converted to TensorFlow Lite format for deployment onto a physical microprocessor environment equipped with a camera module to simulate the smart glasses interface. Results revealed a clear trade-off between classification accuracy and real-time inference performance, with lightweight architectures achieving viable frame rates on the constrained hardware while larger models exceeded its processing capacity. These findings demonstrate that model selection and optimisation are critical design decisions for edge-based plant identification systems and contribute a practical benchmark to inform future development of sustainable, low-power machine vision solutions for precision agriculture and environmental applications.

Keywords – edge computing, machine vision, plant species identification, transfer learning

I. INTRODUCTION

1.1 Background and Motivation

There is a need for constant and consistent innovation within the fields of agriculture and environmental monitoring. Machine vision has become a prominent driver of technological advancement within these areas, allowing for automated analysis of visual data at a scale and speed that traditional methods were unable to achieve. As deep learning techniques have improved, their ability to be used in real-world environmental challenges has expanded. Image-based classification is now used across numerous tasks ranging from crop health monitoring to biodiversity surveying.

As wearable devices are becoming increasingly fashionable within daily usage they represent a promising development within this space. Giving users access to embedded vision systems, hands-free real-time processing becomes possible in areas where conventional computing is impractical. Smart glasses, in particular offer a natural interface for this kind of application, allowing users to identify plant species simply by looking at them.

However, deploying deep learning on wearable hardware proves to be a conflicting task. High-performing neural networks tend to demand equally high-powered computational resources, whereas wearable edge devices are constrained by their limited capacity. Wearable edge devices require low weight control systems and therefore contain limited battery capacity, memory and processing power. It is essential to understand where these constraints become a bottleneck, and which model architectures can operate effectively within it.

Plant species identification is a starting point in the direction of exploring this challenge. Accurate plant identification has direct applications in foraging safety, where misidentification of toxic species poses a genuine risk. As well as in precision agriculture and ecological monitoring, where quick field identification can inform time-sensitive decisions.

1.2 Aims and Objectives

To effectively implement a project to program machine learning models for plant species identification for smart glasses a clearly defined project plan was developed. Creating a structured set of aims and objectives ensured that the project remained focused and that progress could be measured at each stage.

The overall aim of the project was to identify the most effective machine learning model for plant species identification in real-time using smart glasses. Five key objectives were defined to achieve this. First, to collect and prepare a dataset of at least five distinct plant species suitable for model training and evaluation. Second, to train and evaluate at least 3 different machine learning model architectures for plant identification accuracy.

Third, to optimise those models for real-time use on the target smart glasses hardware. Fourth, to configure the smart glasses platform for connection to an external device, for processing in real time. Fifth, to complete a structured comparative analysis of each model's accuracy and real time performance.

1.3 Scope and Limitations

To ensure the project remained achievable within the available time and resources, a defined scope was established. The project includes the selection and comparison of multiple machine learning model architectures, the use of a fixed publicly available image dataset, deployment and testing on a single hardware platform, and the collection of real-time performance metrics.

Several areas were explicitly excluded from the scope. The project does not involve the development of a new machine learning model, the creation of a new plant dataset, or the integration of augmented reality features into the glasses interface. Models were not trained to identify specific plant components such as stems, leaves or bark in isolation. While on-the-fly data augmentation was applied during training to improve model generalisation, this does not constitute the creation of a new dataset the source images remain the publicly available UK Leaf Dataset throughout. These exclusions were made deliberately to keep the comparative analysis controlled and the research question focused.

It should also be noted that a Raspberry Pi microprocessor, rather than the smart glasses hardware directly, was used as the primary processing unit during model evaluation. This acts as a representative proxy, given the shared constraints of ARM-based processors across both platforms, though it is acknowledged as a limitation of the study.

1.4 Report Structure

The report follows a structure as noted. Section II presents a review of existing literature, covering machine vision for plant identification, deep learning architectures suited to edge deployment, and prior work combining smart glasses with plant recognition systems. Section III details the methodology, including dataset preparation, model selection, training, optimisation, and hardware integration. Section IV presents the results of each model's performance and provides a critical discussion of the findings, including a comparative analysis, ethical considerations, limitation and directions for future work. Section V concludes the report with a summary of key findings and the answer to the central research aim.

II. LITERATURE REVIEW

2.1 Machine Vision for Plant Identification

Automated plant species identification has been researched regularly for over a decade, with early

approaches relying on feature extraction algorithms made for specific taxa [1],[3]. These systems needed a great amount of expert botanical knowledge to design and were limited in their generalised use across multiple species. However, the introduction of deep learning, and convolutional neural networks in particular, allowed for models to learn discriminative features directly from image data without manual feature engineering [2],[3].

Barre et al. demonstrated this shift with LeafNet, a CNN based identification system evaluated across the LeafSnap, Flavia and Foliage datasets, showing that learned CNN features consistently outperformed hand-crafted methods for leaf-based species classification [1]. Leaf morphology has since become the dominant choice for plant identification systems, driven by its accessibility. Leaves are typically the most visible and consistently available plant organ in field conditions, making them practical for camera based wearable systems [1]. Lee et al. reinforced this direction by showing that CNNs applied to leaf images automatically identify venation patterns as discriminative interspecies features, providing mechanistic justification for the leaf-based approach beyond empirical accuracy alone [2].

The scalability of deep learning for plant image tasks was further demonstrated by Mohanty et al., who achieved 99.35% classification accuracy across 14 crop species using a CNN trained on over 54,000 images [5]. Although, performance dropped substantially when the same model was tested on real-world images taken outside controlled conditions, highlighting a persistent gap between laboratory accuracy and field deployment performance that remains a challenge in the field [5]. A comprehensive survey by Kumar and Sachar confirms that while deep learning techniques have matured considerably, the challenges of real-world variability, limited training data, and computational constraints continue to motivate ongoing research [4].

These foundations establish leaf-based image classification using CNNs as both technically viable and scientifically supported, providing the methodological basis for the comparative analysis conducted in this study.

2.2 Deep Learning Architectures for Edge Deployment

Convolutional neural networks have become the dominant approach for image classification tasks, with a strong drive in further architectural improvements in both accuracy and efficiency being made [9],[10],[11],[6]. Fundamental challenges arise when deploying these models on resource-constrained hardware. Deeper and wider networks generally achieve higher accuracy, but at a proportionally greater cost in memory, computation and inference time [8]. Understanding this trade-off is very important in selecting appropriate architectures for wearable edge deployment.

Transfer learning has emerged as the standard approach for addressing this challenge in domains where labelled training data is limited [7]. Rather than training a model

from random initialisation, pre-trained weights from large-scale datasets such as ImageNet are used as a starting point, with the final classification layers tuned using the target dataset. This approach significantly reduces the volume of training data required to achieve competitive accuracy and has been validated specifically in the context of plant species identification by Kim and Lee, who showed that transfer learning produces reliable performance evaluation results across multiple architectures [7].

The architectures selected for this study cover a purposeful range of complexity. He et al. introduced ResNet through residual learning framework that uses skip connections to address the vanishing gradient problem in very deep networks, enabling architectures of 50 or more layers to train reliably [9]. ResNet-50 has high classification accuracy but includes a large parameter count and computational footprint. Szegedy et al. developed InceptionV3 using factorised convolutions, replacing computationally expensive large filters with sequences of smaller ones, to extract multi-scale features more efficiently, though the architecture remains substantially heavier than mobile-oriented options [10].

In contrast, Sandler et al. designed MobileNetV2 explicitly for deployment on mobile and resource-constrained hardware, using inverted residuals and depthwise separable convolutions to dramatically reduce computation while preserving representational capacity [11]. The architecture uses ReLU6 activations specifically for robustness under low-precision computation, making it well-suited to INT8 quantisation. Tan and Le introduced EfficientNet through a compound scaling method that systematically balances network depth, width, and input resolution simultaneously, rather than scaling a single dimension in isolation [6]. EfficientNet-B7 achieves state-of-the-art accuracy while being considerably smaller and faster than comparable architectures, and the EfficientNet family has demonstrated strong transfer learning performance on plant classification tasks specifically [6].

Wang et al. provide direct evidence that lightweight architectures are not merely a compromise but a necessary engineering constraint for edge-based plant identification, demonstrating that model size and inference latency must be treated as primary evaluation criteria alongside accuracy when the deployment target is resource-constrained hardware [8]. This establishes the theoretical basis for the comparative evaluation conducted in this study, which tests architectures across this full spectrum of complexity on identical hardware under real-time conditions.

2.3 Plant Identification on Wearable and Mobile Devices

While laboratory plant identification systems have demonstrated strong classification accuracy, deploying these systems on portable and wearable hardware introduces a new set of challenges that are not fully captured by static benchmark results. Mobile phones and

smart glasses share fundamental architectural constraints; ARM-based processors, restricted memory bandwidth, and limited battery capacity. Thus, making mobile-based research a valuable substitute for evaluating wearable deployment feasibility [13].

Rzanny et al. highlighted that image acquisition, and preprocessing decisions have a direct and measurable impact on identification performance, demonstrating that the conditions under which images are captured are as important as the model architecture itself [12]. This finding is particularly relevant to wearable deployment, where camera angle, lighting and motion blur are variable and largely outside the user's control, in contrast to the controlled conditions of laboratory datasets.

Park and Kim applied this research to the smart glasses platform, implementing a plant recognition system using deep learning on wearable glasses hardware and confirming that real-time inference is achievable within the constraints of the wearable form factor [13]. Their work serves as a direct precedent for the hardware configuration adopted in this study and shows that, while significant, the technical challenges of deploying smart glasses can be overcome.

Malik et al.'s EfficientNet-B1 based mobile identification system achieved 87% top-1 accuracy during standard testing but dropped to 78.5% during real-time trials on actual plant samples in natural conditions, further highlighting the discrepancy between controlled-environment accuracy and real-world field performance [14]. The real-time assessment methodology used in this study, which evaluates models under live inference conditions rather than just against a static test set, is explicitly motivated by this accuracy loss under real-world conditions. In a similar vein, research on embedded CNNs for mobile plant recognition shows that ARM processor limitations necessitate careful optimisation choices, especially when choosing depthwise separable convolution architectures, to attain reasonable inference speeds without going over thermal and power budgets.

Collectively, these studies establish that mobile and wearable deployment is not simply a miniaturised version of server-based inference, but a distinct engineering problem requiring hardware-aware model selection, optimisation, and real-world evaluation. This provides the contextual foundation for the comparative analysis conducted in this study.

2.4 Smart Glasses as a Platform for CNN Deployment

Smart glasses represent a uniquely constrained deployment environment for convolutional neural networks. Unlike mobile phones, which can leverage modern ARM processors with dedicated neural processing units, smart glasses must balance computational capability against form factor, thermal dissipation, and battery life within a wearable frame. These constraints place strict upper bounds on the complexity of models capable of operating in real-time [15].

Hu et al. provided evidence that this challenge is tractable, demonstrating a functional CNN-based plant identification system implemented directly on smart glasses [15]. Their work established that the integration of a camera module with an onboard processing unit is architecturally viable for this application, and that the constraints of the wearable form factor, while significant, do not prevent effective deep learning inference. Building on this, Sugi-Ganeshi and Subramani focused on using leaf images for smart glasses-based plant identification. They found that macro-level photos taken from a head-mounted camera contain enough visual detail for CNN classification [16]. Their findings informed the dataset curation in this study, which uses images that mimic the perspective of a wearable camera rather than ideal laboratory conditions.

Ponnusamy et al. further highlight the processing challenges of wearable tech, noting that high-accuracy neural networks usually rely on GPU-level power that smart glasses lack. However, by developing a real-time leaf disease detection system using YOLOv3, they showed that transfer learning can help overcome these hardware limits. This approach allows smaller, faster models to achieve strong classification results without needing to be trained from scratch [17].

The hardware study most comparable to this current work is by González-Huitrón et al. They trained four lightweight CNNs including MobileNetV2, NASNet Mobile, Xception, and MobileNetV3, on a tomato leaf dataset and tested them on a Raspberry Pi 4 for real-time inference. Their work proved that lightweight models could run effectively on single-board computers, though accuracy varies significantly depending on the architecture. This provides a clear basis for the methodology used in this study, which similarly evaluates multiple model architectures on a shared hardware platform [18].

2.5 Research Gap

While previous research has demonstrated the feasibility of deploying CNNs on wearable devices, most of these studies focus on evaluating a single model architecture or prioritise theoretical accuracy over practical deployment metrics. There remains a significant lack of direct, multi-model comparisons conducted on identical, physically constrained hardware under real-time conditions. This project directly addresses this gap by establishing a controlled comparative benchmark, evaluating multiple lightweight CNN architectures against both accuracy and real-world performance metrics on a shared smart glasses hardware platform.

III. METHODOLOGY

3.1 Research Design

A quantitative, experimental comparative methodology was adopted for this study. This approach was selected

because the central research aim required objective, numerical comparison across a set of controlled variables. The independent variable was model architecture, with five distinct architectures evaluated across a consistent set of dependent variables: classification accuracy, precision, recall, inference speed in frames per second, and post-conversion model size. All models were trained on the same dataset, evaluated on the same hardware, and assessed using identical metrics to ensure the validity of comparisons across the architectures.

3.2 Dataset Selection and Preparation

3.2.1 Dataset Acquisition

The UK Leaf Dataset, publicly available on Kaggle, was selected as the primary dataset for training and evaluating the leaf classification models. The dataset contained approximately 340 macro-style images of isolated plant leaves across seven UK species: Maple, Nettle, Hogweed, Blackberry, Blackthorn, Oak, and Laurel. Its photographic style closely resembled the perspective expected from smart-glasses-based image capture, making it well-suited to the ecological identification use case motivating this project. The dataset's open availability also ensured full reproducibility of all experiments conducted.

3.2.2 Dataset Partitioning

Before any preprocessing steps were applied, the dataset was partitioned into three strictly separated subsets to prevent data leakage and ensure unbiased evaluation of model performance. The training, validation, and test subsets comprised 238, 68, and 34 images respectively. The test subset was intentionally kept small relative to the training set to maximise the volume of data available for model learning, while remaining sufficient for statistically meaningful evaluation across seven species classes. This separation ensured that hyperparameter tuning relied exclusively on the validation subset, while final performance metrics were computed using strictly unseen test samples.

3.2.3 Preprocessing Procedures

All images were resized to 224×224 pixels to match the fixed spatial input requirements of the four selected transfer-learning architectures: MobileNetV2, ResNet50, EfficientNetB0, and InceptionV3, each of which was originally trained on ImageNet at this resolution. The custom CNN, trained from random initialisation, did not carry pre-trained weight distributions and was therefore subjected only to the standard resizing and pixel scaling applied uniformly across all inputs.

Pixel normalisation was applied on a per-architecture basis rather than as a uniform transformation across the dataset. Each pre-trained model requires a specific input scaling to remain compatible with its original ImageNet weight distributions. MobileNetV2 and InceptionV3 required pixel values to be scaled to the range -1 to 1 , whereas ResNet50 required RGB-to-BGR colour channel

conversion followed by zero-centring of each channel. These architecture-specific preprocessing functions were implemented using TensorFlow Keras utility layers, applied dynamically within each model's input pipeline to prevent distribution mismatch between incoming pixel values and the pre-trained weight distributions, thereby maintaining gradient stability during fine-tuning.

Augmentation was applied exclusively to the training subset. Validation and test images were subjected only to the resizing and architecture-specific normalisation described above, ensuring that all evaluation metrics reflected model performance on unmodified samples.

3.2.4 Data Augmentation

Given the relatively small size of the original dataset, comprising approximately 340 images and roughly 48 per class, there was a significant risk of overfitting when fine-tuning deep neural networks with parameter counts ranging from approximately 5 million for EfficientNetB0 to 25 million for ResNet50, as well as the custom CNN trained from random initialisation which carried no pre-trained weight advantage. To address this, a data augmentation pipeline was constructed using TensorFlow Keras preprocessing layers and applied dynamically at the tensor level during training.

The pipeline applied random horizontal and vertical flips, random rotations of between -27° and $+27^\circ$, spatial shear transformations, and brightness and zoom adjustments to each image at the point of loading. Performing these operations dynamically rather than saving augmented copies to disk ensured that the model encountered stochastic variations of each training image on every epoch, effectively exposing each architecture to 816 varied image samples across the training process. This approach significantly improved the models' ability to generalise to unpredictable real-world leaf appearances under the variable lighting and camera conditions characteristic of smart-glasses deployment.

3.3 Model Architecture and Training Configuration

3.3.1 Baseline and Transfer Learning Models

To conduct a comprehensive comparative analysis of classification accuracy against edge-hardware performance, five distinct convolutional neural network architectures were trained. A custom lightweight CNN was developed from random initialisation to serve as a performance baseline, providing a reference point against which the benefits of transfer learning could be assessed. This custom architecture comprised three sequential convolutional blocks following a $1/255$ -pixel rescaling layer. The first block utilised a 2D convolutional layer with 32 filters (3×3 kernel) and a ReLU activation function, followed by a 2×2 max pooling layer. Each subsequent block followed this structure while increasing the filter count to 64. To perform the final classification, the spatial dimensions were flattened into a 1D tensor,

passed through a fully connected dense layer of 64 units (ReLU), and mapped to a final dense softmax output layer representing the seven UK leaf classes.

To evaluate the efficacy of transfer learning on a small, domain-specific dataset, four pre-trained architectures were selected: MobileNetV2, EfficientNetB0, ResNet50, and InceptionV3, each initialised with ImageNet weights. MobileNetV2 and EfficientNetB0 were chosen specifically for their edge-optimised designs, which minimise parameter counts and computational latency through architectural techniques such as depthwise separable convolutions and compound scaling respectively, making them primary candidates for real-time inference on the constrained Raspberry Pi hardware. In contrast, ResNet50 and InceptionV3 were included as heavier, industry-standard benchmark architectures, whose greater parameter counts and computational demands were expected to exceed the real-time processing capacity of the deployment hardware, thereby anchoring the upper end of the accuracy spectrum in the comparison.

Unlike the custom baseline CNN, the top classification layers of all four pre-trained models were replaced with a bespoke classification head comprising a global average pooling layer, a dropout layer, and the final 7-class dense softmax output. Global average pooling was preferred over a fully connected flatten layer for these deeper models, as it substantially reduces the parameter count at the classification stage while retaining spatial feature information, mitigating overfitting risk. A dropout rate of 0.2 was applied to introduce stochastic regularisation during training, providing an additional safeguard against overfitting given the limited volume of available training data. The softmax activation function was applied at the output layer to produce a normalised probability distribution across the seven species classes, enabling direct confidence-score comparison between candidate species.

3.3.2 Training Environment and Configuration

Model compilation and training were executed in a cloud-based Python environment using Google Colab with GPU acceleration, which provided the computational resources necessary for fine-tuning the pre-trained architectures within a practical timeframe. All five models were compiled using the Adam optimiser, selected for its adaptive learning rate behaviour, which has been shown to converge reliably across a range of architecture types and is well-suited to fine-tuning scenarios where gradients may vary significantly between layers. Sparse categorical cross-entropy was used as the loss function, as class labels were represented as integers rather than one-hot encoded vectors.

A default Adam learning rate of 0.001, a batch size of 32, and a fixed training duration of 15 epochs were applied consistently across all five architectures. Applying identical hyperparameters across all five models ensured that observed differences in classification performance

were attributable to architectural variation rather than differences in training configuration.

3.3.3 Model Serialisation and TFLite Export

Following training, all models were initially serialised using the modern .keras format, selected over the legacy .h5 format for its improved handling of complex model objects. However, during the subsequent conversion phase to TensorFlow Lite (.tflite), a critical serialisation failure was encountered: when the .keras save file was loaded from disk, Keras was unable to reconstruct the Lambda layers used to implement the architecture-specific preprocessing functions—specifically the RGB-to-BGR channel conversion and zero-centring required by ResNet50, and the custom mathematical scaling required by InceptionV3. This occurred because Keras cannot natively serialise and reconstruct custom Python functions embedded within Lambda layers from a saved file, resulting in a `TypeError` crash during the conversion pipeline.

This failure was systematically resolved by bypassing the saved .keras file entirely for the .tflite conversion step. Instead, the active, in-memory model object—which retained the fully constructed Lambda layers in their original Python state—was passed directly into the TensorFlow Lite converter without being written to and reloaded from disk. This approach circumvented the serialisation limitation while preserving the complete preprocessing pipeline intact through to the edge deployment stage, ensuring that all five models were successfully converted to a deployable format for evaluation on the Raspberry Pi hardware, as detailed in Section 3.4.

3.4 Edge Deployment and Hardware Integration

3.4.1 TFLite Conversion and Operator Compatibility

To deploy the trained convolutional models onto the resource-constrained edge platform, the saved model objects were converted into the TensorFlow Lite (.tflite) format, which is optimised for inference on low-power, memory-limited hardware. During the initial conversion phase, standard post-training quantisation was applied in an attempt to compress the model weights from 32-bit floating point to 8-bit integer (INT8) representation, which would have reduced both model size and inference latency on the target hardware. However, this compressed format produced a .tflite file containing a `FULLY_CONNECTED` operator at version 12, which the version of the tflite-runtime library available on the Raspberry Pi did not support, resulting in a critical runtime failure when the models were loaded on the device.

To resolve this incompatibility, post-training quantisation was explicitly disabled during the conversion process, yielding uncompressed .tflite models whose operator versions were within the supported range of the device's runtime library. This increased the storage footprint of

each exported model relative to what a fully quantised conversion would have produced. The precise storage size of each model before and after conversion is reported in Section 4.3, where the implications of this trade-off are discussed in the context of the broader comparative analysis. Despite the absence of quantisation, all five models were successfully loaded and executed on the Raspberry Pi hardware following this adjustment.

3.4.2 Hardware Integration and Camera Configuration

The edge deployment platform was constructed using a Raspberry Pi single-board computer interfaced with a dedicated camera module, configured to simulate the wearable smart glasses use case described in Section 1.1. Establishing a real-time video inference loop presented a significant software compatibility conflict between the modern libcamera hardware stack used by the current Raspberry Pi OS and the standard OpenCV (cv2) computer vision library, which manifested as memory allocation failures when direct camera access was attempted through the standard VideoCapture interface.

To resolve this conflict, the libcamerify translation wrapper was integrated into the execution environment. This wrapper intercepts calls made by OpenCV and translates them into commands compatible with the libcamera backend, allowing the two systems to interface without requiring modifications to the underlying operating system. The libcamerify wrapper was invoked at the point of script execution using the following command:

```
libcamerify python3 run_glasses.py
```

This solution enabled a stable live camera feed to be captured and processed in real time. Each frame was resized to 224×224 pixels to match the input tensor dimensions required by all five models, and passed through the loaded .tflite model for on-device classification. Inference speed in frames per second and CPU utilisation were logged directly on the device throughout this process, ensuring that all performance benchmarks reflected true on-device conditions under sustained inference load rather than simulated or theoretical estimates. The extent to which each architecture sustained real-time inference within the hardware constraints of the platform is reported in Section 4.3.

3.5 Evaluation Metrics

To objectively assess and compare the five architectures against the core research aim, a structured set of evaluation metrics was established, covering both classification efficacy and edge-hardware viability. During the cloud-based validation phase, overall classification accuracy was recorded alongside per-class precision and recall. Precision and recall were included to provide a more granular assessment of per-species classification reliability than overall accuracy alone, since

a model achieving high aggregate accuracy may still perform poorly on individual species classes. These cloud-based metrics were supplemented by hardware-specific measurements collected during the physical deployment phase. The storage footprint (in Megabytes) of each uncompressed model was measured to assess memory efficiency. Finally, real-time inference speed was quantified in frames per second (FPS) directly on the Raspberry Pi, providing a practical measure of operational latency. The comparative results across these combined metrics form the basis of the analysis presented in Chapter 4.

IV. RESULTS AND DISCUSSION

4.1 Model Training Outcomes

The initial phase of evaluation focused on the learning behaviour and convergence stability of the five architectures during the fixed 15-epoch training cycle. Evaluation of the training and validation curves provided critical insight into how well each model adapted to the augmented dataset, and highlighted any architectural tendencies toward overfitting or underfitting.

The application of transfer learning yielded rapid and highly stable convergence across all four pre-trained models. As illustrated in the comparative validation trajectories (Fig. 1), architectures initialised with ImageNet weights demonstrated an immediate capacity to extract relevant features from the UK leaf dataset, rapidly separating themselves from the baseline model.

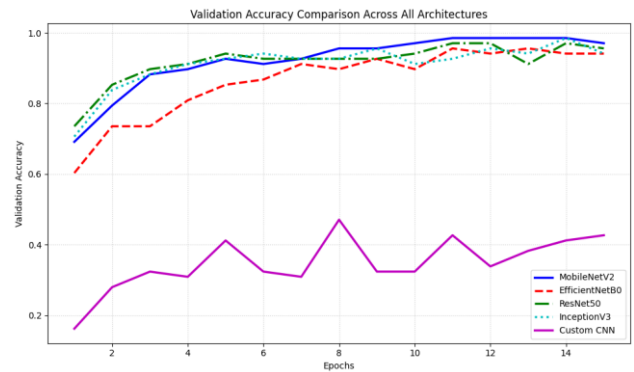


Fig. 1. Comparative validation accuracy across all five architectures over the 15-epoch training cycle.

InceptionV3 achieved the highest final validation accuracy at 99%, indicating robust generalisation. MobileNetV2 (97%) and EfficientNetB0 (96%) exhibited similarly stable trajectories. The convergence of these edge-optimised models within just 15 epochs validates the efficacy of the stochastic data augmentation pipeline (detailed in Section 3.2.4) in preventing the deep architectures from memorising the limited training data. ResNet50 reached a final validation accuracy of 94%; however, its validation loss exhibited increased fluctuation during the final epochs, suggesting the early

onset of overfitting consistent with its larger parameter count.

In stark contrast to the pre-trained networks, the custom-built convolutional neural network struggled significantly to map the underlying feature representations of the dataset. Concluding the 15-epoch training cycle with a final validation accuracy of only 51%, the custom model's individual training curves (Fig. 2) demonstrated a classic case of underfitting.

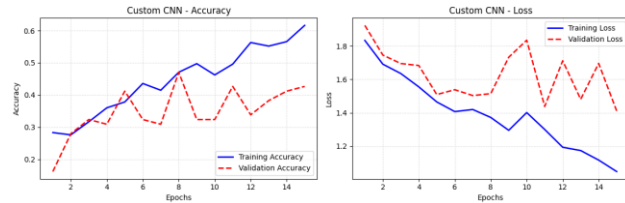


Fig. 2. Training and validation accuracy and loss curves for the custom CNN, demonstrating severe underfitting.

Without the advantage of pre-trained weight distributions, the custom architecture was unable to learn complex spatial hierarchies (such as vein structures and margin serrations) from a random initialisation state within 15 epochs. The visually stark 43-percentage-point accuracy gap (visible in Fig. 1) between the weakest pre-trained model and the custom CNN confirms that transfer learning is not merely beneficial but effectively necessary for achieving competitive classification performance on datasets of this scale. The classification performance of each architecture across the individual species classes is examined in detail in Section 4.2.

4.2 Classification Performance and Error Analysis

While overall accuracy provides a macro-level view of model viability, deploying a visual identification model in a real-world, safety-critical environment requires high reliability across all individual species. A model that achieves high aggregate accuracy but consistently fails to identify a specific poisonous or hazardous plant is fundamentally unfit for purpose. Therefore, a granular analysis of per-class precision and recall was conducted.

As presented in Table I

Model Architecture	Overall Accuracy	Macro Precision	Macro Recall	Macro F1-Score
InceptionV3	99%	0.99	0.98	0.98

Model Architecture	Overall Accuracy	Macro Precision	Macro Recall	Macro F1-Score
MobileNetV2	97%	0.97	0.96	0.96
EfficientNet B0	96%	0.98	0.96	0.96
ResNet50	94%	0.96	0.93	0.94
Custom CNN	51%	0.63	0.49	0.46

, InceptionV3, MobileNetV2, and EfficientNetB0 all maintained macro-averaged F1-scores of 0.96 or higher. A deeper inspection of their classification reports revealed highly consistent identification across the majority of the dataset. MobileNetV2, the leading candidate for edge deployment, achieved perfect recall (1.00) on six of the seven species classes.

However, cross-referencing the per-class metrics across all architectures isolated Hogweed as the most challenging species to classify. MobileNetV2 experienced its only significant performance drop on this class, recording a recall of 0.75, indicating that 25% of the Hogweed validation images were misclassified as other species. Even the heaviest, most accurate model, InceptionV3 (99% overall accuracy), recorded its lowest recall score (0.88) on Hogweed. ResNet50, despite its 94% overall accuracy, exhibited the most uneven per-class distribution of the pre-trained models, consistent with the overfitting tendency observed in its training curves. This systemic difficulty across multiple distinct convolutional architectures strongly implies that Hogweed shares overlapping morphological features—such as leaf shape, vein patterns, or margin serrations—with another species in the dataset, creating a difficult decision boundary for the classifiers.

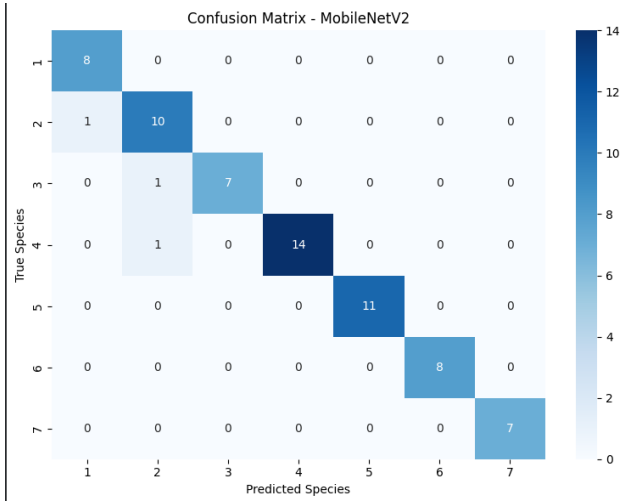


Fig. 3. Confusion matrix for MobileNetV2, detailing precise prediction distributions across the 7 species classes.

This morphological overlap presented a far greater challenge for the baseline custom CNN. Lacking the sophisticated feature-extraction layers of the pre-trained models, the custom CNN suffered severely degraded classification performance on several individual classes. Its recall for Hogweed fell to just 0.12, meaning it successfully identified the species in only 12% of cases. It additionally recorded recall scores of only 0.25 for Maple and 0.27 for Nettle.

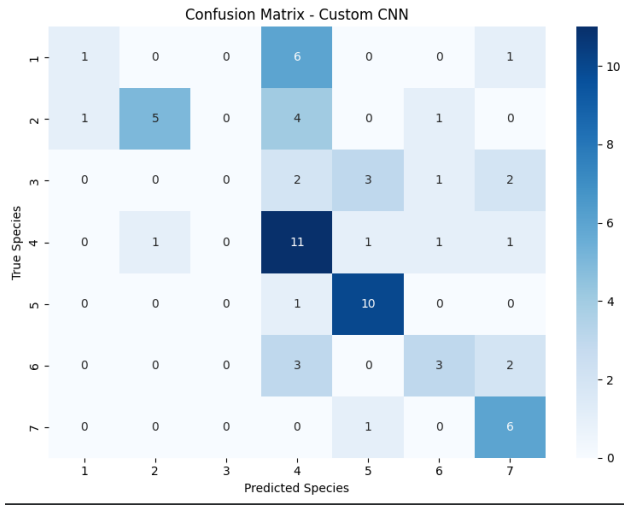


Fig. 4. Confusion matrix for the custom CNN, highlighting severe misclassification rates and class confusion.

Analysis of the custom CNN's confusion matrix (Fig. 4) reveals that rather than learning distinct, generalisable features for these complex classes, the model frequently defaulted its predictions to the remaining classes, further confirming the underfitting diagnosed in Section 4.1. These findings demonstrate that only the pre-trained architectures achieved the per-class reliability necessary

for safe deployment in real-world ecological identification contexts.

4.3 Edge Deployment Viability: Hardware Benchmarking

While Section II established the theoretical classification accuracy of the architectures, deploying these models onto wearable hardware introduces severe computational constraints. For the smart glasses to be practical for real-world foraging, the software would need to process live video feeds with minimal latency without overwhelming the Raspberry Pi's memory or thermal limits. To evaluate real-world viability, each converted .tflite model was benchmarked directly on the Raspberry Pi. Table 4.2 presents the measured inference speed, CPU utilisation, and resident memory consumption for each architecture under sustained live inference conditions on the Raspberry Pi.

Table II: Hardware Benchmarking Results on Raspberry Pi

Model Architecture	Inference Speed (FPS)	CPU Utilisation (%)	RAM Usage (RES)	Performance Category
Custom CNN	~12.5 FPS	262%	116 MB	Fastest, but inaccurate
MobileNetV2	~8.0 FPS	196%	118 MB	Viable for Deployment
EfficientNetB0	~3.3 FPS	148%	139 MB	Below real-time threshold
InceptionV3	~1.9 FPS	278%	269 MB	Unsuitable for Real-Time use
ResNet50	~1.4 FPS	271%	286 MB	Unsuitable for Real-Time use

The hardware data decisively eliminates the heaviest models from deployment consideration. Despite achieving 99% validation accuracy, InceptionV3 proved entirely unsuitable for edge deployment, consuming 269 MB of RAM and bottlenecking the video feed to just 1.9 FPS. ResNet50 performed worse, drawing the highest RAM footprint (286 MB) and rendering a near-unusable 1.4 FPS. At these speeds, the user interface would experience extreme latency, rendering the smart glasses unresponsive and impractical for active environmental scanning. Furthermore, both models forced the CPU utilisation to near 280%, pushing the CPU toward its thermal limits on the passively cooled Raspberry Pi platform.

In contrast, MobileNetV2 demonstrated exceptional hardware efficiency. Operating at a stable 8.0 FPS, MobileNetV2 provided a sufficiently responsive

identification experience for the field use case, processing a new frame approximately every 125 milliseconds a latency acceptable for the deliberate, stationary scanning behaviour expected in a foraging context. Crucially, it achieved this while maintaining a lightweight footprint of just 118 MB of RAM and a manageable 196% CPU load, leaving over half of the Pi's processing capacity free for background operating system tasks and camera rendering.

While the baseline custom CNN achieved the highest throughput at 12.5 FPS, its severe underfitting and 51% accuracy (diagnosed in Section 4.1) mean its speed advantage carries no practical value given the model's inability to classify reliably. EfficientNetB0, despite its edge-focused design, underperformed expectations on this specific hardware configuration, achieving only 3.3 FPS. This suggests that EfficientNetB0's architectural optimisations may not translate efficiently to the ARM-based processor architecture of the Raspberry Pi, warranting further investigation.

Ultimately, the hardware benchmarking validates MobileNetV2 as the only architecture capable of bridging the gap between high-precision plant species classification (97% accuracy) and the strict computational realities of wearable edge devices.

4.4 Comparative Analysis and Final Architecture Selection

Selecting the optimal neural network for a wearable ecological device requires balancing a strict trade-off between classification precision and edge-hardware efficiency. No single architecture excelled across all isolated metrics, necessitating a holistic comparative analysis to determine the final deployment model.

The custom CNN was immediately eliminated; while it achieved the highest inference speed (12.5 FPS), its profound underfitting and severe class confusion (51% accuracy) rendered it unsuitable for any practical plant identification application. The heavyweight models (InceptionV3 and ResNet50) represented the opposing extreme. While InceptionV3 achieved a near-perfect 99% validation accuracy, its massive computational footprint bottlenecked the Raspberry Pi to 1.9 FPS, failing the latency requirements of a real-world wearable device.

The viable candidates were therefore narrowed to the edge-optimised architectures: EfficientNetB0 and MobileNetV2. While both achieved excellent classification metrics (96% and 97% accuracy, respectively), MobileNetV2 demonstrated markedly superior hardware compatibility, achieving 8.0 FPS compared to EfficientNetB0's 3.3 FPS. EfficientNetB0 struggled to translate its theoretical efficiency to the ARM-based processor, whereas MobileNetV2 delivered its highly stable frame rate while consuming minimal

RAM (118 MB), leaving sufficient overhead for the operating system.

When per-class reliability was additionally considered, specifically MobileNetV2's consistent recall across the majority of species classes, including hazardous ones, the comparative analysis firmly established MobileNetV2 as the optimal architecture. It was the only model tested that successfully bridged the gap between the high-accuracy requirements of ecological safety and the strict thermal and computational limits of wearable edge hardware. Consequently, MobileNetV2 was selected as the final model for the smart glasses deployment platform, and its performance implications are discussed further in Chapter 5.

4.5 Ethical Considerations and Real-World Safety

The physical deployment of computer vision for ecological foraging introduces significant ethical and safety considerations that standard classification metrics alone fail to capture. While aggregate accuracy indicates technical proficiency, the real-world application of this technology demands that not all misclassifications carry equal risk. An analysis of the system's failure modes is critical to ensuring user safety.

The primary ethical concern lies in the asymmetrical risk of false positives and false negatives when identifying hazardous species. As identified in Section 4.2, the architectures struggled to isolate Hogweed—a species whose sap causes severe phototoxic burns upon skin contact. If the system misidentifies a safe species (such as Maple) as Hogweed, the consequence is merely a missed foraging opportunity. However, if the system confidently misidentifies Hogweed as a benign species, it actively facilitates physical harm to the user. The morphological overlap that caused the 25% recall drop for Hogweed in MobileNetV2 highlights the inherent danger of relying solely on deep learning for safety-critical ecological identification without secondary warning systems.

Beyond misclassifying known species, the system must also safely handle out-of-distribution (OOD) data. In a real-world environment, the wearable camera will inevitably process foliage and objects entirely absent from its limited seven-class training distribution. To evaluate the model's epistemic uncertainty—the model's inability to distinguish between known classes when confronted with unfamiliar inputs—MobileNetV2 was subjected to physical OOD testing on the Raspberry Pi, wherein the camera was presented with unlearned environmental objects. During this stress test, the architecture's predictive confidence degraded appropriately, outputting highly uncertain probability distributions ranging between 40% and 50% for its top predictions.

This confidence collapse is a highly desirable safety characteristic for edge deployment. Because the model demonstrates appropriate uncertainty when confronted

with unknown variables, the risk of confident misidentification can be programmatically mitigated. By implementing a strict confidence threshold within the edge software—for example, requiring a minimum confidence of 80% before displaying a classification to the user—the smart glasses can effectively filter out the 40–50% OOD predictions. Consequently, the system will fail safely by returning an "Unknown Species" state, preventing the user from acting on dangerously uncertain AI output. While this confidence thresholding mechanism was not implemented within the scope of the current prototype, its integration represents a critical prerequisite for any production deployment of this system in a safety-critical context, and is identified as a priority direction for future development in Chapter 5.

4.6 Limitations and Future Work

While the on-device deployment of MobileNetV2 successfully eliminates the latency and energy costs associated with cloud-based inference, the current prototype remains constrained by its restricted training distribution. The reliance on a seven-class dataset of primarily static, well-lit leaf images does not fully capture the dynamic visual conditions of a live foraging environment, where motion blur, variable canopy lighting, and considerable species diversity introduces significant noise. Consequently, the model's performance boundaries regarding severe morphological overlap, particularly concerning hazardous species like Hogweed, require further empirical validation across a broader and more ecologically diverse dataset.

Future development should prioritise expanding this training distribution to encompass a comprehensive regional flora database. Any such expansion would need to be carefully managed to ensure that model complexity remains within the thermal and memory constraints of the target wearable hardware. These expansions to the training distribution should be accompanied by the confidence thresholding mechanism proposed in Section 4.5, which represents the most immediately actionable safety improvement to the current prototype. At a system level, the integration of GPS localisation data could enable contextual species probability weighting based on known geographic distribution and seasonal occurrence, substantially reducing the classifier's effective decision space in any given field environment. Implementing these systemic safeguards will ultimately evolve the prototype from an isolated classifier into a robust, safety-critical ecological tool.

V. CONCLUSIONS

5.1 Summary of Findings

This research evaluated the viability of deploying deep convolutional neural networks onto wearable edge hardware for real-world ecological identification. The empirical evaluation revealed a clear and significant trade-

off between theoretical classification accuracy and real-time hardware inference performance.

While the heavyweight architectures—InceptionV3 and ResNet50—achieved the highest validation accuracies during cloud-based training, they exceeded the processing capacity of the wearable hardware. Their deployment resulted in critically degraded frame rates (below 2 FPS) and unsustainable thermal and memory loads, rendering them entirely unsuitable for real-time applications.

Conversely, the lightweight architectures demonstrated that edge-based machine vision is physically viable when models are specifically engineered for constrained environments. MobileNetV2 and EfficientNetB0 successfully navigated the hardware limitations, though MobileNetV2 emerged as the decisively superior architecture. It achieved a stable 8.0 FPS inference speed while maintaining a highly reliable 97% classification accuracy. Ultimately, the findings confirm that raw model accuracy is an insufficient metric for wearable AI; rigorous model selection, architectural efficiency, and hardware-aware model conversion are critical design decisions for successful on-device plant identification.

5.2 Answer to the Research Aim

The primary aim of this research was to determine the most effective neural network architecture for real-time, edge-based ecological identification, balancing the competing demands of accuracy, inference speed, and physical hardware limits.

This study has successfully answered that aim by identifying MobileNetV2 as the optimal architecture for the specified wearable platform. Through a multi-criteria evaluation encompassing macro-level accuracy, hazardous species recall, out-of-distribution safety behaviour, and strict CPU/RAM benchmarking, MobileNetV2 was demonstrated to be the only model capable of delivering safe, responsive, and computationally efficient classifications.

Beyond identifying a single optimal model, this dissertation contributes a comprehensive, empirical hardware benchmark to the broader academic field. By detailing the exact performance degradation of standard convolutional networks when transitioned from cloud environments to ARM-based edge processors, this study provides a practical baseline for future researchers developing safety-critical, on-device machine vision systems for ecological and agricultural applications.

VI. REFERENCES

- [1] P. Barré, B. C. Stöver, K. F. Müller, and V. Steinhage, "LeafNet: A computer vision system for automatic plant species identification," *Ecol. Inform.*, vol. 40, pp. 50–56, Jul. 2017.
- [2] S. H. Lee, C. S. Chan, P. Wilkin, and P. Remagnino, "Deep-plant: Plant identification with convolutional neural networks," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2015, pp. 452–456.
- [3] S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, "Deep neural networks based recognition of plant diseases by leaf image classification," *Comput. Intell. Neurosci.*, vol. 2016, pp. 1–11, 2016.
- [4] A. Kumar and S. Sachar, "Plant leaf disease detection using deep learning: A review," *Multimed. Tools Appl.*, vol. 82, no. 14, pp. 21457–21491, 2023.
- [5] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, p. 1419, Sep. 2016.
- [6] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.
- [7] W. S. Kim and S. K. Lee, "Performance evaluation of plant species identification using deep learning models," *J. Korea Multimed. Soc.*, vol. 23, no. 3, pp. 446–455, 2020.
- [8] J. Wang, Y. Liu, and H. Zheng, "A light-weight deep learning model for plant identification," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, 2022, pp. 1823–1828.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [10] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2818–2826.
- [11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.
- [12] M. Rzanny, M. Seeland, J. Wäldchen, and P. Mäder, "Acquiring and preprocessing leaf images for automated plant identification: Understanding the influence of image characteristics on classification performance," *Plant Methods*, vol. 13, no. 1, p. 93, 2017.
- [13] S. Park and Y. Kim, "A plant recognition system on wearable smart glasses using deep learning," in *Proc. 11th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, 2019, pp. 293–295.
- [14] Malik, O., Ismail, N., Hussein, B. and Yahya, U. (2022). Automated Real-Time Identification of Medicinal Plants Species in Natural Environment Using Deep Learning Models—A Case Study from Borneo Region. *Plants*, 11.
- [15] H. Hu, J. Wang, and W. Li, "Design of a smart glasses based on CNN for plant identification," in *Proc. Int. Conf. Intell. Informatics Biomed. Sci. (ICIIBMS)*, 2019, pp. 238–241.
- [16] S. Sugi-Ganeshi and B. Subramani, "An application of deep learning for plant identification based on leaf images on smart glasses," in *Proc. 5th Int. Conf. Devices, Circuits Syst. (ICDCS)*, 2020, pp. 166–169.
- [17] R. Ponnusamy, N. Rajkumar, and C. Krishna Kumar, "Leaf disease detection and classification using deep learning algorithm," in *Proc. Int. Conf. Innov. Comput. Commun. (ICICC)*, vol. 1087, 2020, pp. 507–516.
- [18] V. González-Huitrón, J. A. León-Borges, A. Rodríguez-Mata, L. E. Amabilis-Sosa, B. Ramírez-Pereda, and H. Rodríguez, "Disease detection in plants using deep learning," *IEEE Lat. Am. Trans.*, vol. 19, no. 6, pp. 1015–1023, Jun. 2021.